only—if you use the HTML analogy, some QML components are like <*script*> tags and are not rendered on the screen.

If you do want to create a widget, you need to inherit from *QQuickPaintedItem*. This base class has a lot of glue logic that's needed for painting on the screen already implemented, and it's easier to get access to the OpenGL context through this.

If you don't need screen access (like us), you can simply derive from *QObject* or if you want some of the glue taken care of, you can derive from *QQuickItem*. *QQuickPaintedItem* derives from this and adds the screen real estate management logic, but does handle screen events; so if you want a component that triggers certain actions within the application based on GUI events (without being visible), you need to derive from *QQuickItem*. We won't need any of the special functionality when we implement the fortune client, but we'll derive from *QQuickItem* anyway.

The code

Let's begin. This is what the FortuneClient class looks like:

```
#include <QQuickItem>
#include <QString>
#include <QByteArray>
#include <QTcpSocket>

class FortuneClient : public QQuickItem
{
    Q_OBJECT
    Q_PROPERTY(QString serverHost READ serverHost WRITE
setServerHost)
    Q_PROPERTY(int     serverPort READ serverPort WRITE
setServerPort)

    private:

    quint16 mPort;
    QString mHost;

    QTcpSocket * clientSocket;

    public:

    FortuneClient();
    ~FortuneClient();

    QString serverHost() const;
    void setServerHost(QString host);
    quint16 serverPort() const;
    void setServerPort(int port);

    signals:

    void haveFortune(QString fortune);

    public slots:

    void getNewFortune();
};
```

There's a lot that's important here.

Let's begin with the *Q_PROPERTY* macros, which tell the MOC to expose properties to the QML environment. The minimal syntax looks like this:

```
Q_PROPERTY(property_type property_name READ read_function
WRITE write_function)
```

So now you'll know that the first *Q_PROPERTY* line exposes a *QString* property, called *serverHost*, to the QML environment. To read this property, the *serverHost()* function is called, and to write to this property, the *setServerHost()* function is called.

Now let's take a look at how those functions are implemented. The reader function, *serverHost()*, is declared *const* and simply returns a *QString*. That's all you need to do - declare your function *const*, which means telling the compiler this function doesn't change the state of the object, and return some data of the property type.

Now you'll have to do a little hackery with integer data. QML has an integer data type, and C++ has many integer data types, depending on how many bits you want to use. When you're specifying a *Q_PROPERTY*, however, you can only use a standard *int* as its type, since all the different *qint* and *quint* types don't have equivalents in the QML environment. So your handler functions must take care to check ranges and cast them into properly-sized integers.

The other very important bit that you need to know is how the signal and slot mechanisms work with QML.

First of all, there are two ways in which you can execute a C++ function from within the QML environment. The first is to precede a function declaration with the *Q_INVOKABLE* macro, as follows:

```
Q_INVOKABLE void someFunction(QString some_arg);
```

This makes the function visible from the QML environment. All functions, even public ones, are not visible from the QML environment, due to security concerns.

The second way is to simply declare the function as a public slot. All public slots are visible from the QML environment. And that's what we have done here with *getNewFortune()*—declared it as a slot.

Now for the signals. When you define signals that are to be accessed from the QML environment, you must stick to some naming conventions. You must use *camelCase()*, with the initial letter being lower case. This is because in the QML environment, the signal gets turned into a property, with the name *onCamelCase*, to which you assign a function, JavaScript or C++, which gets executed when the signal is emitted. For example,